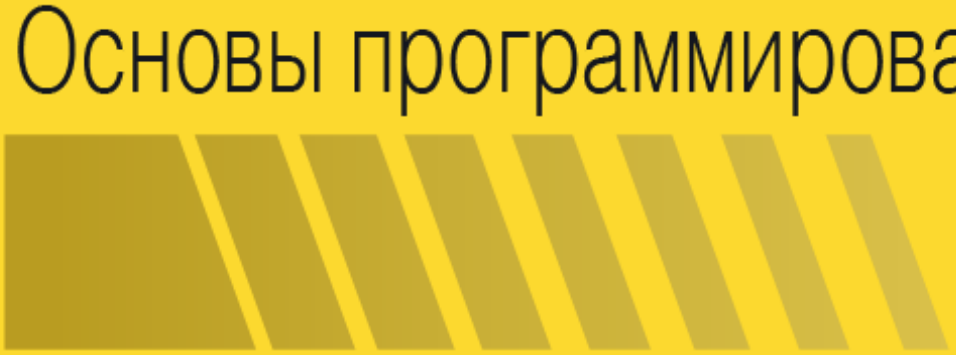


COURSE

Основы программирования



JS

Функции

shesnokov@gmail.com

ОГЛАВЛЕНИЕ

План лекции *3*

Теоретическая часть *3*

Определение функции. **3**

Вызов функций **5**

Аргументы и параметры функций **7**

Списки аргументов переменной длины **7**

Параметры по умолчанию **10**

Деструктуризация параметров **11**

Свойства и методы функций **12**

Свойство name. 12

Свойство length. 12

Свойство prototype. 13

Методы call() и apply(). 13

Метод bind(). 14

Стрелочные функции. **15**

Литература и другие источники *19*

Вопросы для самопроверки *19*

ПЛАН ЛЕКЦИИ

1. Определение функции;
2. Вызов функций;
3. Аргументы и параметры функций;
4. Списки аргументов переменной длинны;
5. Параметры по умолчанию;
6. Деструктуризация параметров;
7. Свойства и методы функций;
8. Стрелочные функции.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

ОПРЕДЕЛЕНИЕ ФУНКЦИИ.

Функция - это блок программного кода на языке JavaScript, который определяется один раз и может выполняться, или вызываться, многократно. Возможно, вы уже знакомы с понятием «функция» под другим названием, таким как ***подпрограмма***, или ***процедура***. Функции могут иметь параметры: определение функции может включать список идентификаторов, которые называются параметрами и играют роль локальных переменных в теле функции.

При вызове функций им могут передаваться значения, или аргументы, соответствующие их параметрам. Функции часто используют свои аргументы для вычисления возвращаемого значения, которое является значением выражения вызова функции. В дополнение к аргументам при вызове любой функции ей передается еще одно значение, определяющее контекст вызова - значение в ключевом слове this.

Функции в языке JavaScript являются ***объектами*** и могут использоваться разными способами. Например, ***функции могут присваиваться переменным и передаваться другим функциям***. Поскольку функции являются объектами, имеется возможность присваивать значения их свойствам и даже вызывать их методы.

В JavaScript допускается создавать определения функций, вложенные в другие функции, и такие функции будут иметь доступ ко всем переменным, присутствующим в области видимости определения.

Определение функции начинается с ключевого слова function, за которым указываются следующие компоненты:

1. ***Идентификатор, определяющий имя функции*** - имя является обязательной частью инструкции объявления функции: оно будет использовано для создания новой переменной, которой будет присвоен объект новой функции. В выражениях определения функций имя может отсутствовать: при его наличии имя будет ссылаться на объект функции только в теле самой функции.
2. ***Пара круглых скобок вокруг списка из нуля или более идентификаторов, разделенных запятыми*** - эти идентификаторы будут определять ***имена параметров функции*** и в теле функции могут использоваться как локальные переменные.
3. ***Пара фигурных скобок с нулем или более инструкций JavaScript внутри*** - эти инструкции составляют ***тело функции***: они выполняются при каждом вызове функции.

В следующем примере показано несколько определений функций в виде инструкций и выражений.

Обратите внимание, что определения функций в виде выражений удобно использовать, только если они являются частью более крупных выражений, таких как присваивание или вызов функции, которые выполняют некоторые действия с помощью вновь объявленной функции:

```
// Выводит имена и значения всех свойств объекта obj
```

```
function printprops(obj) {
        for(var p in obj)
                console.log(p + ": " + obj[p] + "\n");
}

// Вычисляет расстояние между точками (x1,y1) и (x2,y2)
function distance(x1, y1, x2, y2) {
        var dx = x2 - x1;
        var dy = y2 - y1;
        return Math.sqrt(dx*dx + dy*dy);
}
// Рекурсивная функция (вызывающая сама себя), вычисляющая факториал
function factorial(x) {
        if (x <= 1) return 1;
        return x * factorial(x-1);
}

// Следующее выражение определяет функцию, вычисляющую
// квадрат аргумента. Обратите внимание,
// что она присваивается переменной
var square = function(x) { return x*x; }

// Выражения определения функций могут иметь имена, что позволяет
// производить рекурсивные вызовы
var f = function fact(x) {
        if (x <= 1)
                return 1;
        else
                return x*fact(x-1);
};

// Выражения определения функций иногда могут тут же вызываться:
var tensquared = (function(x) {return x*x;}(10));
```

Обратите внимание, что в выражениях определения функций имя функции может отсутствовать. Инструкция объявления функции фактически объявляет переменную и присваивает ей объект функции.

Выражение определения функции, напротив, не объявляет переменную. Однако в выражениях определения допускается указывать имя функции, как в функции вычисления факториала выше, которое может потребоваться в теле функции для вызова себя самой. Если выражение определения функции включает имя, данное имя будет ссылаться на объект функции в области видимости этой функции.

Фактически имя функции становится локальной переменной, доступной только в теле функции. В большинстве случаев имя функции не требуется указывать в выражениях определения, что делает определения более компактными.

Обратите внимание, что большинство (но не все) функций в примере содержат инструкцию return. Инструкция return завершает выполнение функции и выполняет возврат значения своего выражения (если указано) вызывающей программе. Если выражение в инструкции return отсутствует, она возвращает значение undefined. Если инструкция return отсутствует в функции, интерпретатор просто выполнит все инструкции в теле функции и вернет вызывающей программе значение undefined.

Большинство функций в примере вычисляют некоторое значение, и в них инструкция return используется для возврата этого значения вызывающей программе. Функция printprops() несколько отличается в этом смысле: ее работа заключается в том, чтобы вывести имена свойств объекта. Ей не нужно возвращать какое-либо значение, поэтому в функции отсутствует инструкция return. Функция printprops() всегда будет возвращать значение undefined. (Функции, не имеющие возвращаемого значения, иногда называются процедурами.)

Вызов функций

Программный код, образующий тело функции, выполняется не в момент определения функции, а в момент ее вызова. Вызов функций выполняется с помощью выражения вызова. Выражение вызова состоит из выражения обращения к функции, которое возвращает объект функции, и следующими за ним круглыми скобками со списком из нуля или более выражений-аргументов, разделенных запятыми, внутри.

Если функция является свойством объекта или элементом массива (т.е. методом) - тогда выражение вызова является ***выражением вызова метода***. В следующем фрагменте демонстрируется несколько примеров выражений вызова обычных функций:

```
printprops({x:4, age: 24});
var d = distance(1,1,5,6);
var f = factorial(5) / factorial(12);
f = square(5);
```

При вызове функции вычисляются все выражения-аргументы (указанные между скобками), и полученные значения используются в качестве аргументов функции. Эти значения присваиваются параметрам, имена которых перечислены в определении функции. В теле функции выражения обращений к параметрам возвращают значения соответствующих аргументов.

При вызове обычной функции возвращаемое функцией значение становится значением выражения вызова. Если возврат из функции происходит по достижении ее конца интерпретатором, возвращается значение undefined. Если возврат из функции происходит в результате выполнения инструкции return, возвращается значение выражения, следующего за инструкцией return, или undefined, если инструкция return не имеет выражения.

Метод - это не что иное, как функция, которая хранится в виде свойства объекта. Если имеется функция func и объект obj, то можно определить метод объекта obj с именем method, как показано ниже:

```
// Определим простой объект и функцию
var obj = {};
function func(a, b) { return a+b;}
```

```
// Добавим в объект obj метод
obj.method = func;
// Теперь можно вызвать этот метод
var result = obj.method(4, 5);
```

Чаще всего при вызове методов используется форма обращения к свойствам с помощью оператора точки, однако точно так же можно использовать форму обращения к свойствам с помощью квадратных скобок. Например, оба следующих выражения являются выражениями вызова методов:

```
result = obj.method(4, 5);
result = obj['method'](4, 5);
```

Аргументы и возвращаемое значение при вызове метода обрабатываются точно так же, как при вызове обычной функции. Однако вызов метода имеет одно важное отличие: контекст вызова. Выражение обращения к свойству состоит из двух частей: объекта (в данном случае obj) и имени свойства (method). В подобных выражениях вызова методов объект obj становится контекстом вызова, и тело функции получает возможность ссылаться на этот объект с помощью ключевого слова this. Например,

```
var obj = {
        x: 0,
        y: 0,

        // Метод
        add: function(a, b) {
                this.x = a;
                this.y = b;
        },

        // Еще один метод
        sum: function() {
                return this.x + this.y
        }
};

// Вызов методов
obj.add(15, 4);
console.log(obj.sum());    // 19
```

Методы и ключевое слово this занимают центральное место в парадигме объектно-ориентированного программирования. Любая функция, используемая как метод, фактически получает неявный аргумент - объект, относительно которого она была вызвана. Как правило, методы выполняют некоторые действия с объектом, и синтаксис вызова метода наглядно отражает тот факт, что функция оперирует объектом.

Обратите внимание: this - это именно ключевое слово, а не имя переменной или свойства. Синтаксис JavaScript не допускает возможность присваивания значений элементу this.

Аргументы и параметры функций

В языке JavaScript, в определениях функций не указываются типы параметров, а при вызове функций не выполняется никаких проверок типов передаваемых значений аргументов. Фактически при вызове функций в языке JavaScript не проверяется даже количество аргументов.

Когда число аргументов в вызове функции меньше числа объявленных параметров, недостающие аргументы получают значение undefined. Часто бывает удобным писать функции так, чтобы некоторые аргументы были необязательными и могли опускаться при вызове функции. В этом случае желательно предусмотреть возможность присваивания достаточно разумных значений по умолчанию параметрам, которые могут быть опущены. Например,

```
// Добавить в массив arr перечислимые имена
// свойств объекта obj и вернуть его. Если аргумент
// arr не был передан, создать и вернуть новый массив
function getPropertyNames(obj, /* необязательный */ arr) {
        if (arr === undefined)
                arr = [];               // Если массив не определен, создать новый
        for(var property in obj)
                arr.push(property);
        return arr;
}

// Эта функция может вызываться с 1 или 2 аргументами:
var a = getPropertyNames({x:1, y:1});         // Получить свойства объекта в новом массиве
getPropertyNames({z:5},a);                    // добавить свойства нового объекта в этот массив
console.log(a);         // ['x', 'y', 'z']
```

Обратите внимание, что при объявлении функций необязательные аргументы должны завершать список аргументов, чтобы их можно было опустить. Программист, который будет писать обращение к вашей функции, не сможет передать второй аргумент и при этом опустить первый: он будет вынужден явно передать в первом аргументе значение undefined. Обратите также внимание на комментарий /* необязательный */ в определении функции, который подчеркивает тот факт, что параметр является необязательным.

Списки аргументов переменной длины

Если число аргументов в вызове функции превышает число имен параметров, функция лишается возможности напрямую обращаться к неименованным значениям. Решение этой проблемы предоставляет объект Arguments. В теле функции идентификатор arguments ссылается на объект Arguments, присутствующий в вызове. Объект Arguments - это объект, подобный массиву, позволяющий извлекать переданные функции значения по их номерам, а не по именам.

Предположим, что была определена функция func, которая требует один аргумент x. Если вызвать эту функцию с двумя аргументами, то первый будет доступен внутри функции по имени параметра x

или как arguments[0]. Второй аргумент будет доступен только как arguments[1]. Кроме того, подобно настоящим массивам, arguments имеет свойство length, определяющее количество содержащихся элементов. То есть в теле функции func, вызываемой с двумя аргументами, arguments.length имеет значение 2.

Объект Arguments может использоваться с самыми разными целями. Следующий пример показывает, как с его помощью проверить, была ли функция вызвана с правильным числом аргументов, - ведь JavaScript этого за вас не сделает:

```
function func(x, y, z) {
        // Сначала проверяется, правильное ли количество аргументов передано
        if (arguments.length != 3) {
                throw new Error("Функция func вызвана с " + arguments.length
                        + " аргументами, а требуется 3.");
        }

        // А теперь сам код функции...
}
```

Зачастую нет необходимости проверять количество аргументов, как в данном примере. Поведение по умолчанию интерпретатора JavaScript отлично подходит для большинства случаев: отсутствующие аргументы замещаются значением undefined, а лишние аргументы просто игнорируются.

Объект Arguments иллюстрирует важную возможность JavaScript-функций: они могут быть написаны таким образом, чтобы работать с любым количеством аргументов. Следующая функция принимает любое число аргументов и возвращает значение самого большого из них (аналогично ведет себя встроенная функция Math.max()):

```
function maxNumber() {
        var m = Number.NEGATIVE_INFINITY;
        // Цикл по всем аргументам, поиск и
        // сохранение наибольшего из них
        for(var i = 0; i < arguments.length; i++)
                if (arguments[i] > m)
                        m = arguments[i]; // Вернуть наибольшее значение
        return m;
}
var largest = maxNumber(1, 10, 100, 2, 3, 1000, 4, 5, 10000, 6);                    // 10000
```

Функции, подобные этой и способные принимать произвольное число аргументов, называются ***функциями с переменным числом аргументов (variadic functions, variable arity functions или varargs functions)***. Этот термин возник вместе с появлением языка программирования C.

Обратите внимание, что функции с переменным числом аргументов не должны допускать возможность вызова с пустым списком аргументов. Будет вполне разумным использовать объект arguments[]

при написании функции, ожидающей получить фиксированное число обязательных именованных аргументов, за которыми может следовать произвольное число необязательных неименованных аргументов.

Не следует забывать, что arguments фактически не является массивом - это объект Arguments. В каждом объекте Arguments имеются пронумерованные элементы массива и свойство length, но с технической точки зрения это не массив. Лучше рассматривать его как объект, имеющий некоторые пронумерованные свойства.

Помимо элементов своего массива объект Arguments определяет свойства callee и caller. При попытке изменить значения этих свойств в строгом режиме ECMAScript 5 гарантированно возбуждается исключение TypeError. Однако в нестрогом режиме стандарт ECMAScript утверждает, что свойство callee ссылается на выполняемую в данный момент функцию. Свойство caller не является стандартным, но оно присутствует во многих реализациях и ссылается на функцию, вызвавшую текущую.

Свойство caller можно использовать для доступа к стеку вызовов, а свойство callee особенно удобно использовать для рекурсивного вызова неименованных функций:

```
var factorial = function(x) {
        if (x <= 1) return 1;
        return x * arguments.callee(x-1);
};
```

Чтобы получить массив аргументов в функции, можно использовать **оператор …**, например,

```
function showName(firstName, lastName, ...rest) {
        alert(firstName + ' ' + lastName + ' - ' + rest);
}
// выведет: Юлий Цезарь - Император,Рима
showName("Юлий", "Цезарь", "Император", "Рима");
```

В rest попадёт массив всех аргументов, начиная с третьего. Следует отметить, что rest – настоящий массив, с методами map, forEach и другими, в отличие от arguments. Оператор … должен быть размещен в конце. Оператор … собирает «все оставшиеся» аргументы, поэтому такое объявление не имеет смысла - параметр ...rest должен быть в конце функции.

```
function f(arg1, ...rest, arg2) { // arg2 после ...rest ?!
  // будет ошибка
}
```

Этот же оператор можно использовать и при вызове функции, для передачи массива параметров как списка, например,

```
'use strict';
let numbers = [2, 3, 15];

// Оператор ... в вызове передаст массив как список аргументов
// Этот вызов аналогичен Math.max(2, 3, 15)
let max = Math.max(...numbers);
```

```
alert( max ); // 15
```

Формально, эти два вызова делают одно и то же:

```
Math.max(...numbers);
Math.max.apply(Math, numbers);
```

Параметры по умолчанию

В функциях можно указывать параметры по умолчанию через равенство =, например,

```
function showMenu(title = "Без заголовка", width = 100, height = 200) {
	alert(title + ' ' + width + ' ' + height);
}
showMenu("Меню"); // Меню 100 200
```

Параметр по умолчанию используется при отсутствующем аргументе или равном undefined, например,

```
function showMenu(title = "Заголовок", width = 100, height = 200) {
	alert('title=' + title + ' width=' + width + ' height=' + height);
}

// По умолчанию будут взяты 1 и 3 параметры
// title=Заголовок width=null height=200
showMenu(undefined, null);
```

При передаче любого значения, кроме undefined, включая пустую строку, ноль или null, параметр считается переданным, и значение по умолчанию не используется.

Параметры по умолчанию могут быть не только значениями, но и выражениями. Например,

```
function sayHi(who = getCurrentUser().toUpperCase()) {
	alert('Привет, ' + who);
}

function getCurrentUser() {
	return 'Вася';
}
sayHi(); // Привет, ВАСЯ
```

Заметим, что значение выражения getCurrentUser().toUpperCase() будет вычислено, и соответствующие функции вызваны – лишь в том случае, если это необходимо, то есть когда функция вызвана без параметра.

В частности, выражение по умолчанию не вычисляется при объявлении функции. В примере выше функция getCurrentUser() будет вызвана именно в последней строке, так как не передан параметр.

Деструктуризация параметров

Если функция получает объект, то она может его тут же разбить в переменные:

```
'use strict';

let options = {
	title: "Меню",
	width: 100,
	height: 200
};

function showMenu({title, width, height}) {
	alert(title + ' ' + width + ' ' + height); // Меню 100 200
}

showMenu(options);
```

Можно использовать и более сложную деструктуризацию, с соответствиями и значениями по умолчанию:

```
'use strict';
let options = {
	title: "Меню"
};
function showMenu({title="Заголовок", width:w=100, height:h=200}) {
	alert(title + ' ' + w + ' ' + h);
}
// объект options будет разбит на переменные
showMenu(options); // Меню 100 200
```

Заметим, что в примере выше какой-то аргумент у showMenu() обязательно должен быть, чтобы разбить его на переменные. Чтобы функция могла быть вызвана вообще без аргументов – нужно добавить ей параметр по умолчанию – уже не внутрь деструктуризации, а в самом списке аргументов:

```
'use strict';
function showMenu({title="Заголовок", width:w=100, height:h=200} = {}) {
	alert(title + ' ' + w + ' ' + h);
}
showMenu(); // Заголовок 100 200
```

В коде выше весь объект аргументов по умолчанию равен пустому объекту {}, поэтому всегда есть что деструктурировать.

Свойства и методы функций

В JavaScript- функции могут использоваться как значения. Оператор typeof возвращает для функций строку «function», однако в действительности функции в языке JavaScript - это особого рода объекты. А раз функции являются объектами, то они имеют свойства и методы, как любые другие объекты. Существует даже конструктор Function(), который создает новые объекты функций. В следующих подразделах описываются свойства и методы функций.

Свойство name.

В свойстве name у функции находится её имя. Например,

```
'use strict';
function f() {} // f.name == "f"
let g = function g() {}; // g.name == "g"
alert(f.name + ' ' + g.name) // f g
```

В примере выше показаны Function Declaration и Named Function Expression. В синтаксисе выше довольно очевидно, что у этих функций есть имя name. В конце концов, оно указано в объявлении. Но современный JavaScript идёт дальше, он старается даже анонимным функциям дать разумные имена.

Например, при создании анонимной функции с одновременной записью в переменную или свойство – её имя равно названию переменной (или свойства).

```
'use strict';
// свойство g.name = "g"
let g = function() {};
let user = {
        // свойство user.sayHi.name == "sayHi"
        sayHi: function() {}
};
```

Свойство length.

В теле функции свойство arguments.length определяет количество аргументов, переданных функции. Однако свойство length самой функции имеет иной смысл. Это свойство, доступное только для чтения, возвращает количество аргументов, которое функция ожидает получить - число объявленных параметров.

В следующем фрагменте определяется функция с именем check(), получающая массив аргументов arguments от другой функции. Она сравнивает свойство arguments.length (число фактически переданных аргументов) со свойством arguments.callee.length (число ожидаемых аргументов), чтобы определить, передано ли функции столько аргументов, сколько она ожидает. Если значения не совпадают, генерируется исключение. За функцией check() следует тестовая функция func(), демонстрирующая порядок использования функции check():

```
// Эта функция использует arguments.callee, поэтому она
// не будет работать в строгом режиме
function check(args) {
        var actual = args.length;                // Фактическое число аргументов
```

```
        var expected = args.callee.length;      // Ожидаемое число аргументов

        if (actual !== expected)                // Если не совпадают, генерируется исключение
                throw new Error("ожидается: " + expected + "; получено " + actual);
}

function func(x, y, z) {
        // Проверить число ожидаемых и фактически переданных аргументов
        check(arguments);

        // Теперь выполнить оставшуюся часть функции
        return x + y + z;
}
```

Свойство prototype.

Любая функция имеет свойство prototype, ссылающееся на объект, известный как объект прототипа. Каждая функция имеет свой объект прототипа. Когда функция используется в роли конструктора, вновь созданный объект наследует свойства этого объекта прототипа.

Методы call() и apply().

Методы call() и apply() позволяют выполнять косвенный вызов функции, как если бы она была методом некоторого другого объекта. Первым аргументом обоим методам, call() и apply(), передается объект, относительно которого вызывается функция; этот аргумент определяет контекст вызова и становится значением ключевого слова this в теле функции. Чтобы вызвать функцию func() (без аргументов) как метод объекта obj, можно использовать любым из методов, call() или apply():

```
func.call(obj);
func.apply(obj);
```

Любой из этих способов вызова эквивалентен следующему фрагменту (где предполагается, что объект obj не имеет свойства с именем m):

```
obj.m = func;           // Временно сделать func методом obj
obj.m();                // Вызывать его без аргументов.
delete obj.m;           // Удалить временный метод.
```

В строгом режиме ECMAScript 5 первый аргумент методов call() и apply() становится значением this, даже если это простое значение, null или undefined. В ECMAScript 3 и в нестрогом режиме значения null и undefined замещаются глобальным объектом, а простое значение - соответствующим объектом-оберткой.

Все остальные аргументы метода call(), следующие за первым аргументом, определяющим контекст вызова, передаются вызываемой функции. Метод apply() действует подобно методу call(), за исключе-

нием того, что аргументы для функции передаются в виде массива. Если функция способна обрабатывать произвольное число аргументов, метод apply() может использоваться для вызова такой функции в контексте массива произвольной длины.

В следующем примере демонстрируется практическое применение метода call():

```
// Ниже определены две функции, отображающие свойства и
// значения свойств произвольного объекта. Способ
// отображения передаются в виде аргумента func
function print1(func, obj) {
        for (n in obj) func(n +': ' + obj[n]);
}

function print2(func, objDevice, obj) {
        for (n in obj) func.call(objDevice, n +': ' + obj[n]);
}

var obj = {x:5, y:10};
print2(document.write, document, obj);  // Работает корректно
print2(console.log, console, obj);
print1(document.write, obj);        // Возникнет исключение Illegal invocation, т.к.
print1(console.log, obj);           // невозможно вызвать эти методы без объекта контекста
```

Метод bind().

Метод bind() впервые появился в ECMAScript 5, но его легко имитировать в ECMAScript 3. Как следует из его имени, основное назначение метода bind() состоит в том, чтобы связать (bind) функцию с объектом. Если вызвать метод bind() функции func и передать ему объект obj, он вернет новую функцию. Вызов новой функции (как обычной функции) выполнит вызов оригинальной функции func как метода объекта obj. Любые аргументы, переданные новой функции, будут переданы оригинальной функции. Например,

```
// Функция, которую требуется привязать
function func(y) {
        return this.x + y;
}

var obj = {x:1};            // Объект, к которому выполняется привязка
var g = func.bind(obj);     // Вызов g(x) вызовет obj.func(x)

Такой способ связывания легко реализовать в ECMAScript 3, как показано ниже:

// Возвращает функцию, которая вызывает func как метод объекта obj
```

```
// и передает ей все свои аргументы
function bind(func, obj) {
        if (func.bind)
                return func.bind(obj); // Использовать метод bind, если имеется
        else return function() {
                // Иначе связать, как показано ниже
                return func.apply(obj, arguments);
        };
}
```

Метод bind() в ECMAScript 5 не просто связывает функцию с объектом. Он также выполняет частичное применение: помимо значения this связаны будут все аргументы, переданные методу bind() после первого его аргумента. Частичное применение - распространенный прием в функциональном программировании и иногда называется ***каррингом (currying)***.

Стрелочные функции.

Появился новый синтаксис для задания функций через «стрелку» => (***стрелочные функции***). Его простейший вариант выглядит так:

```
'use strict';
let inc = x => x+1;
alert( inc(1) ); // 2
Эти две записи – примерно аналогичны:
let inc = x => x+1;
let inc = function(x) { return x + 1; };
```

Как видно, "x => x+1" – это уже готовая функция. Слева от => находится аргумент, а справа – выражение, которое нужно вернуть. Если аргументов несколько, то нужно обернуть их в скобки, вот так:

```
'use strict';
let sum = (a,b) => a + b;
// аналог с function
// let sum = function(a, b) { return a + b; };
alert( sum(1, 2) ); // 3
```

Если нужно задать функцию без аргументов, то также используются скобки, в этом случае – пустые:

```
'use strict';
// вызов getTime() будет возвращать текущее время
let getTime = () => new Date().getHours() + ':' + new Date().getMinutes();
alert( getTime() ); // текущее время
```

Когда тело функции достаточно большое, то можно его обернуть в фигурные скобки {…}:

```
'use strict';
let getTime = () => {
```

```
        let date = new Date();
        let hours = date.getHours();
        let minutes = date.getMinutes();
        return hours + ':' + minutes;
};
alert( getTime() ); // текущее время
```

Заметим, что как только тело функции оборачивается в {…}, то её результат уже не возвращается автоматически. Такая функция должна делать явный return, как в примере выше, если конечно хочет что-либо возвратить.

Функции-стрелки очень удобны в качестве коллбэков, например,

```
`use strict`;
let arr = [5, 8, 3];
let sorted = arr.sort( (a,b) => a - b );
alert(sorted); // 3, 5, 8
```

Такая запись – коротка и понятна. Далее мы познакомимся с дополнительными преимуществами использования функций-стрелок для этой цели. Функции-стрелки не имеют своего this. Внутри функций-стрелок – тот же this, что и снаружи. Это очень удобно в обработчиках событий и коллбэках, например,

```
'use strict';
let group = {
        title: "Наш курс",
        students: ["Вася", "Петя", "Даша"],
        showList: function() {
                this.students.forEach(
                student => alert(this.title + ': ' + student)
                )
        }
}

group.showList();
// Наш курс: Вася
// Наш курс: Петя
// Наш курс: Даша
```

Здесь в forEach была использована функция-стрелка, поэтому this.title в коллбэке – тот же, что и во внешней функции showList. То есть, в данном случае – group.title. Если бы в forEach вместо функции-стрелки была обычная функция, то была бы ошибка:

```
'use strict';
let group = {
        title: "Наш курс",
        students: ["Вася", "Петя", "Даша"],
```

```
        showList: function() {
                this.students.forEach(function(student) {
                        alert(this.title + ': ' + student); // будет ошибка
                })
        }
}
group.showList();
```

При запуске будет "попытка прочитать свойство title у undefined", так как .forEach(f) при запуске f не ставит this. То есть, this внутри forEach будет undefined.

Функции стрелки нельзя запускать с new. Отсутствие у функции-стрелки "своего this" влечёт за собой естественное ограничение: такие функции нельзя использовать в качестве конструктора, то есть нельзя вызывать через new.

Есть тонкое различие между функцией стрелкой => и обычной функцией, у которой вызван .bind(this):

- вызовом .bind(this) мы передаём текущий this, привязывая его к функции.
- при => привязки не происходит, так как функция стрелка вообще не имеет контекста this. Поиск this в ней осуществляется так же, как и поиск обычной переменной, то есть, выше в замыкании. До появления стандарта ES-2015 такое было невозможно.

Функции-стрелки не имеют своего arguments. В качестве arguments используются аргументы внешней «обычной» функции. Например,

```
'use strict';
function f() {
        let showArg = () => alert(arguments[0]);
        showArg();
}
f(1); // 1
```

Вызов showArg() выведет 1, получив его из аргументов функции f. Функция-стрелка здесь вызвана без параметров, но это не важно: arguments всегда берутся из внешней «обычной» функции. Сохранение внешнего this и arguments удобно использовать для форвардинга вызовов и создания ***декораторов***.

Например, декоратор defer(f, ms) ниже получает функцию f и возвращает обёртку вокруг неё, откладывающую вызов на ms миллисекунд:

```
'use strict';
function defer(f, ms) {
        return function() {
                setTimeout(() => f.apply(this, arguments), ms)
        }
}
```

```
function sayHi(who) {
	alert('Привет, ' + who);
}
let sayHiDeferred = defer(sayHi, 2000);
sayHiDeferred("Вася"); // Привет, Вася через 2 секунды
```

Аналогичная реализация без функции-стрелки выглядела бы так:

```
function defer(f, ms) {
	return function() {
		let args = arguments;
		let ctx = this;
		setTimeout(function() {
			return f.apply(ctx, args);
		}, ms);
	}
}
```

В этом коде пришлось создавать дополнительные переменные args и ctx для передачи внешних аргументов и контекста через замыкание.

ЛИТЕРАТУРА И ДРУГИЕ ИСТОЧНИКИ

Самый лучший общедоступный ресурс с точки зрения простоты изложения материала – это «Современный учебник по Java Script», доступный по ссылке https://learn.javascript.ru/books .

Еще один учебник Антон Кармазин «Выразительный JavaScript», доступен по ссылке https://www.gitbook.com/book/karmazzin/eloquentjavascript_ru/details .

Руководство, которое можно использовать в качестве карманного справочника - http://bonsaiden.github.io/JavaScript-Garden/ru/ .

Данных ресурсов достаточно, чтобы получить начальные навыки владения языком программирования.

Современные печатные издания:

1. Закас Н. JavaScript для профессиональных веб-разработчиков / (Пер. с англ. А. Лютича). – СПб.: Питер, 2015. – 960с.
2. Флэнаган Д. JavaScript. Подробное руководство, 6-е издание. – Пер. с англ. – СПб: Символ-Плюс, 2012. – 1080 с., ил.
3. Васильев А.Н. JavaScript в примерах и задачах. / Алексей Васильев. – М.: Издательство «Э», 2017. – 720 с.
4. Дакетт, Джон. JavaScript и jQuery. Интерактивная веб-разработка / Джон Дакетт ; (пер. с англ. М.А. Райтмана). – М.: Издательство «Э»217. – 640 с.

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

1. Дайте определение понятию «функция». Какие особенности задания функций присутствуют в JavaScript?
2. Приведите примеры определения функций.
3. Как в функцию передаются параметры?
4. Можно ли при вызове функции использовать переменное число параметров? Приведите пример.
5. Для чего в определении функции используется оператор … ?
6. Как в функции определить параметры по умолчанию?
7. Что такое деструктуризация параметров функции?
8. Какие стандартные свойства и методы имеются у функций в JavaScript?
9. Как определяются стрелочные функции? Приведите примеры.